Web Application Development

Lecture 3: Components, Props & Events

SWE 381 – Web Application Development

Dr. Ohoud Alharbi Dr. Achraf Gazdar

Department of Software Engineering
College of Computer and Information Sciences
King Saud University

Term 2 – 2025/2026

Learning Objectives

By the end of this lecture, you will be able to:

- State the **rules** of functional components in React
- Pass and read **props** from parent to child components
- Apply **destructuring** and **default values** to props
- Explain why **props are read-only** and what that means
- Use the **children** prop to build wrapper components
- Handle user actions with **onClick**, **onChange**, and **onSubmit**
- Pass functions as props using the **callback props pattern**
- Organise a React project with a clean **folder structure**

Outline

What is a Functional Component?

W3Schools – React Components

A React component is like a **JavaScript function**. It accepts inputs called **props** and returns React elements (JSX) describing what should appear on the screen. When creating a React component, the component's name **MUST start with an upper case letter**.

```
// Simplest functional component
function Car() {
  return <h2>I am a Car!</h2>;
}

// Arrow function style (same result)
const Car = () => {
  return <h2>I am a Car!</h2>;
};

// Use it as a custom HTML tag
createRoot(document.getElementById('root'))
  .render(<Car />);
```

Browser Output

I am a Car!

Component = Function + JSX

A component is just a JavaScript function that **returns JSX**. React calls it, renders the JSX, and puts the result in the DOM.

Functional Component Rules – W3Schools

The 5 Rules (W3Schools):

1. Name **must start with a capital letter**
   `function Car()` not `function car()`
2. Must **return JSX** (or `null`)
3. Can only return **one top-level element**
   Wrap siblings in `<div>` or `<>...</>`
4. Must be a **pure function** with respect to props
   Same props → same output; never modify props
5. Always **export** it for use in other files
   `export default Car`

```jsx
// Rule 1: Capital name
function CourseCard() {  // OK!

  const code = "SWE381";

  // Rule 2: Returns JSX
  // Rule 3: One top-level element
  return (
    <div>
      <h2>{code}</h2>
      <p>Web App Dev</p>
    </div>
  );
}

// Rule 5: Export for reuse
export default CourseCard;
```

Common mistake

Lowercase names like `<car />` are treated as HTML tags, not React components. Always capitalise!

Components Are Reusable

W3Schools – React Components

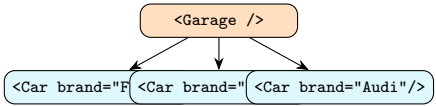
React is all about re-using code. You can use a component as many times as you want, each time with different data.

```
function Car(props) {
  return <h2>I am a {props.brand}!</h2>;
}

// Reuse the same component 3 times
function Garage() {
  return (
    <>
      <h1>Who lives in my Garage?</h1>
      <Car brand="Ford" />
      <Car brand="BMW" />
      <Car brand="Audi" />
    </>
  );
}

createRoot(document.getElementById('root'))
  .render(<Garage />);
```

Browser Output

Who lives in my Garage?

I am a Ford!
I am a BMW!
I am a Audi!



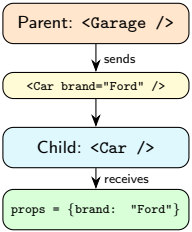
Outline

What Are Props?

W3Schools – React Props

Props are arguments passed into React components. Props are passed to components via HTML attributes. React Props are like **function arguments** in JavaScript and **attributes** in HTML.

```jsx
// Receiving props as an object
function Car(props) {
  return (
    <h2>I am a {props.brand}!</h2>
  );
}

// Pass props like HTML attributes
function Garage() {
  return (
    <>
      <h1>My Garage</h1>
      <Car brand="Ford" />
    </>
  );
}
```



Note: The object is named `props` by convention, but you can call it anything: `function Car(myobj)` works too.

Passing Multiple Props – All Data Types

```jsx
function Car(props) {
  return (
    <div>
      <h2>{props.brand} {props.model}</h2>
      <p>Color: {props.color}</p>
      <p>Year:  {props.year}</p>
      <p>Price: {props.price}</p>
      <p>Electric: {
        props.isElectric ? "Yes" : "No"
      }</p>
    </div>
  );
}

// Strings use quotes
// Numbers, booleans, objects use { }
function App() {
  return (
    <Car
      brand="Tesla"
      model="Model S"
      color="red"
      year={2024}
      price={89000}
      isElectric={true}
    />
  );
}
```

Browser Output

Tesla Model S
Color: red
Year: 2024
Price: 89000
Electric: Yes

W3Schools prop types:

Type	Syntax
String	`brand="Ford"`
Number	`year={2024}`
Boolean	`electric={true}`
Variable	`name={varName}`
Object	`car={{...}}`
Array	`items={[...]}`
Function	`onClick={fn}`

Outline

Props Destructuring – W3Schools

W3Schools – Props Destructuring

React uses curly brackets to destructure props. This way, the component receives all the properties, but the destructuring makes sure it only uses the ones it needs.

Without destructuring (verbose):

```
function Greeting(props) {
  return (
    <h1>
      Hello, {props.name}!
      You are {props.age} years old.
    </h1>
  );
}
```

With parameter destructuring (clean):

```
// Destructure directly in signature
function Greeting({ name, age }) {
  return (
    <h1>
      Hello, {name}!
      You are {age} years old.
    </h1>
  );
}

createRoot(document.getElementById('root'))
  .render(<Greeting name="John" age={25} />);
```

Destructure inside the body:

```
// Or destructure inside the function
function Car(props) {
  const { brand, model } = props;
  return (
    <h2>I love my {brand} {model}!</h2>
  );
}

createRoot(document.getElementById('root'))
  .render(
    <Car brand="Ford"
         model="Mustang"
         color="red"
         year={1969} />
  );
```

Output

I love my Ford Mustang!

Prefer destructuring

Destructuring removes repetitive `props.` prefixes and makes component signatures self-documenting.

Default Prop Values – W3Schools

W3Schools – Props Destructuring

With Destructuring, you can set **default values** for props. If a property has no value, the default value will be used.

```
// Set default value with = in destructuring
function Car({ color = "blue", brand }) {
  return (
    <h2>My {color} {brand}!</h2>
  );
}

createRoot(document.getElementById('root'))
  .render(<Car brand="Ford" />);
// color not provided => uses "blue"
```

Output – no color passed

My blue Ford!

```
createRoot(document.getElementById('root'))
  .render(<Car brand="Tesla" color="red" />);
// color provided => overrides default
```

Output – color passed

My red Tesla!

Multiple defaults:

```
function StudentCard({
  name,
  gpa  = 0.0,
  dept = "Undeclared",
  year = 1
}) {
  return (
    <div>
      <h3>{name}</h3>
      <p>GPA:  {gpa}</p>
      <p>Dept: {dept}</p>
      <p>Year: {year}</p>
    </div>
  );
}

// Minimal call -- all defaults apply
<StudentCard name="Ahmed" />
```

Output

Ahmed
GPA: 0
Dept: Undeclared
Year: 1

The Rest Operator in Props

W3Schools – Props Destructuring

When you don't know how many properties you will receive, you can use the `...rest` operator. You can specify the properties you need, and the rest will be stored in an object.

```
// Destructure specific props, capture everything else in 'rest'
function Car({ color, brand, ...rest }) {
  return (
    <h2>My {brand} {rest.model} is {color}!</h2>
  );
  // rest = { model: "Mustang", year: 1969 }
}

createRoot(document.getElementById('root'))
  .render(<Car brand="Ford" model="Mustang" color="red" year={1969} />);
```

Output

My Ford Mustang is red!

Common use-case: forwarding remaining props

The rest operator is useful when building **wrapper components** that forward extra props to the underlying HTML element without listing them all explicitly.

Outline

Props Are Read-Only – W3Schools Rule

W3Schools – React Props

Note: React Props are read-only! You will get an error if you try to change their value.

```
// WRONG -- modifying props causes error
function Car({ brand }) {
  brand = "Changed!"; // ERROR!
  return <h2>{brand}</h2>;
}
```

Cannot assign to read-only property

React will throw an error or silently fail if you attempt to modify a prop. Props are **immutable** from the child's point of view.

Why are props read-only?

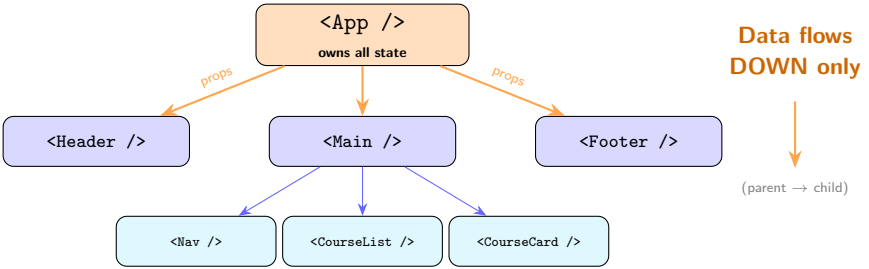
- Ensures **predictable** data flow (parent controls data)
- Makes components **pure** – same inputs, same output
- Prevents **hidden side effects** in child components
- Enables React's **efficient reconciliation**

```
// CORRECT -- use local state for changes
import { useState } from 'react';

function Car({ brand }) {
  // Initialise local state from prop
  const [color, setColor] =
    useState("blue");

  return (
    <div>
      <h2>{color} {brand}</h2>
      <button onClick={() =>
        setColor("red")
      }>
        Paint Red
      </button>
    </div>
  );
}
```

Rule of thumb

Props come from the parent (read only). **State** is managed inside the component (mutable).

Props Flow – Unidirectional Data Flow



Props flow downward

Props always flow from **parent to child**. A child can never push data up using props.

Callbacks push data up

To communicate **child to parent**, the parent passes a **callback function** as a prop (covered in section 5).

Outline

The children Prop – W3Schools

W3Schools – Props Children

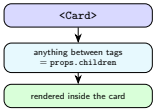
children is a special React prop. The children prop is whatever is placed between the component's opening and closing tags.

```jsx
// Card is a generic container
function Card({ children }) {
  return (
    <div className="card"
      style={{ border: '1px solid #ccc',
               padding: '16px',
               borderRadius: '8px' }}>
      {children}
    </div>
  );
}

// Use Card with any content
function App() {
  return (
    <>
      <Card>
        <h2>SWE 381</h2>
        <p>Web Application Development</p>
      </Card>

      <Card>
        <img src="logo.png" alt="KSU" />
        <p>King Saud University</p>
      </Card>
    </>
  );
}
```

Browser Output

SWE 381
Web Application Development

[logo.png]
King Saud University



children + Other Props Together

```jsx
// Section component: title prop + children
function Section({ title, children }) {
  return (
    <section>
      <h2 style={{ borderBottom: '2px solid #61dafb' }}>
        {title}
      </h2>
      <div className="section-body">
        {children}
      </div>
    </section>
  );
}

// Button with children as label
function Button({ onClick, children }) {
  return (
    <button onClick={onClick}
      style={{ padding: '8px 16px',
               background: '#61dafb' }}>
      {children}
    </button>
  );
}

function App() {
  return (
    <Section title="SWE 381">
      <p>Welcome to React!</p>
      <Button onClick={() => alert('Hi!')}>
        Say Hello
      </Button>
    </Section>
  );
}
```

Browser Output

SWE 381

Welcome to React!

Say Hello

Common `children` use cases:

- **Layout** wrappers (`Card`, `Panel`, `Modal`)
- **Buttons** – label is the children
- **Navigation** items
- Generic **containers** with styling

children is flexible

`children` can be a string, JSX, a component, or even an array of elements.

Outline

React Events Overview – W3Schools

W3Schools – React Events

Just like HTML DOM events, React can perform actions based on user events. React has the same events as HTML: click, change, mouseover, etc. React events are written in **camelCase** syntax: `onClick` instead of `onclick`.

HTML event	React event	Trigger	Common use
`onclick`	`onClick`	Mouse click	Buttons, links
`onchange`	`onChange`	Input value change	Text fields, selects
`onsubmit`	`onSubmit`	Form submission	Login, search forms
`onmouseover`	`onMouseOver`	Hover	Tooltips
`onkeydown`	`onKeyDown`	Key pressed	Shortcuts
`onfocus`	`onFocus`	Element focused	Form fields

React events vs HTML events

In HTML: `onclick="shoot()"` (string). In React: `onClick={shoot}` (function reference in curly braces – **no quotes, no parentheses**).

onClick – W3Schools Example

Basic click handler:

```
function Football() {
  const shoot = () => {
    alert("Great Shot!");
  }

  return (
    <button onClick={shoot}>
      Take the shot!
    </button>
  );
}
```

Passing arguments to handler:

```
function Football() {
  const shoot = (a) => {
    alert(a);
  }

  return (
    // Wrap in arrow fn to pass args
    <button onClick={() => shoot("Goal!")}>
      Take the shot!
    </button>
  );
}
```

Accessing the event object:

```
function Football() {
  const shoot = (a, b) => {
    alert(b.type);
    // 'click'
  }

  return (
    <button onClick={
      (event) => shoot("Goal!", event)
    }>
      Take the shot!
    </button>
  );
}
```

Do NOT call the function

`onClick={shoot}` – passes the function.
`onClick={shoot()}` – calls it immediately on render!
Use `onClick={() => shoot("arg")}` to pass arguments.

onChange – Controlled Inputs

W3Schools – React Forms

We can use the `useState` Hook to keep track of each input value and provide a single source of truth. The input element uses the `onChange` attribute to detect changes.

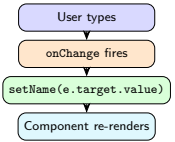
```jsx
import { useState } from 'react';

function MyForm() {
  const [name, setName] = useState("");

  function handleChange(e) {
    // e.target.value is the new text
    setName(e.target.value);
  }

  return (
    <form>
      <label>
        Enter your name:
        <input
          type="text"
          value={name}
          onChange={handleChange}
        />
      </label>
      <p>
        You entered: <b>{name}</b>
      </p>
    </form>
  );
}
```

As user types "Ahmed"

Enter your name:

Ahmed

You entered: **Ahmed**



onSubmit – Handling Form Submission

```jsx
import { useState } from 'react';

function LoginForm() {
  const [email,    setEmail]    = useState("");
  const [password, setPassword] = useState("");

  // Prevent default browser page reload, then handle data
  function handleSubmit(e) {
    e.preventDefault();           // stop browser reload!
    console.log("Email:", email, "Password:", password);
    alert(`Logging in as ${email}`);
  }

  return (
    <form onSubmit={handleSubmit}>
      <label>Email:
        <input type="email" value={email}
               onChange={e => setEmail(e.target.value)} />
      </label>
      <br />
      <label>Password:
        <input type="password" value={password}
               onChange={e => setPassword(e.target.value)} />
      </label>
      <br />
      <button type="submit">Log In</button>
    </form>
  );
}
```

Always call `e.preventDefault()` in onSubmit

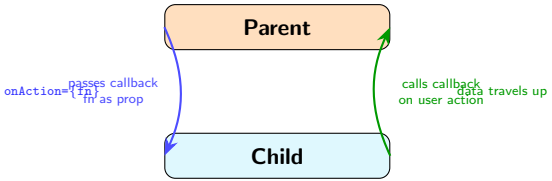
Without it, the browser reloads the page on every form submit, losing all React state.

Outline

The Callback Props Pattern

Problem: Props flow *down* only. How can a child tell its parent something happened?

Solution: The parent passes a **function as a prop**. The child calls it when needed.



This pattern is used for:

- Buttons that trigger parent actions
- Forms where parent owns the data
- Deleting items from a list
- Selecting/toggling items
- Any child-to-parent communication

Naming convention

Callback props are named `on` + `EventName`: `onClick`, `onChange`, `onDelete`, `onSelect`, `onClose`

Callback Props – Basic Example

```jsx
// Child: receives a callback as prop
function ShootButton({ onShoot }) {
  return (
    <button onClick={() => onShoot("Goal!")}>
      Take the Shot!
    </button>
  );
}

// Parent: defines the callback,
//         owns the state
import { useState } from 'react';

function Football() {
  const [message, setMessage] =
    useState("Waiting...");

  // This function is passed as prop
  const handleShoot = (result) => {
    setMessage(result);
  };

  return (
    <div>
      <p>{message}</p>
      <ShootButton onShoot={handleShoot} />
    </div>
  );
}
```

Before click

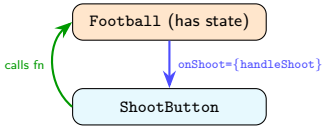
Waiting...

Take the Shot!

After click

Goal!

Take the Shot!



Callback Props – Delete Item Example

```jsx
// Child: one list item with a delete button
function StudentItem({ student, onDelete }) {
  return (
    <li>
      {student.name} -- GPA: {student.gpa}
      <button onClick={() => onDelete(student.id)}>
        Delete
      </button>
    </li>
  );
}

// Parent: manages the students array
import { useState } from 'react';

function StudentList() {
  const [students, setStudents] = useState([
    { id: 1, name: 'Ahmed',  gpa: 3.8 },
    { id: 2, name: 'Sara',   gpa: 3.5 },
    { id: 3, name: 'Khalid', gpa: 3.2 },
  ]);

  const handleDelete = (id) => {
    setStudents(students.filter(s => s.id !== id));
  };

  return (
    <ul>
      {students.map(s => (
        <StudentItem key={s.id} student={s} onDelete={handleDelete} />
      ))}
    </ul>
  );
}
```

Callback Props – Complete Form Example

```jsx
// Reusable input component
function TextInput({ label, value, onChange }) {
  return (
    <div>
      <label>{label}</label>
      <input
        type="text"
        value={value}
        onChange={e => onChange(e.target.value)}
      />
    </div>
  );
}

// Parent owns state, passes callbacks
import { useState } from 'react';

function RegistrationForm() {
  const [name,  setName]  = useState("");
  const [email, setEmail] = useState("");

  function handleSubmit(e) {
    e.preventDefault();
    alert(`Registered: ${name} (${email})`);
  }

  return (
    <form onSubmit={handleSubmit}>
      <TextInput label="Name:"
        value={name} onChange={setName} />
      <TextInput label="Email:"
        value={email} onChange={setEmail} />
      <button type="submit">Register</button>
    </form>
  );
}
```

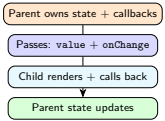
Browser Output

Name:
Ahmed

Email:
ahmed@ksu.edu.sa

Register

Pattern summary:



Outline

Why Folder Structure Matters

W3Schools – React Components

It can be a good idea to split your components into **separate files**. To do that, create a new file in the `src` folder with a `.jsx` file extension and put the code inside it. The filename **must start with an uppercase character**.

Without organisation:

```
src/
  App.jsx     <- everything here!
  main.jsx
```

Problems as the app grows:

- One massive file – hard to navigate
- Cannot reuse components easily
- Merge conflicts in team projects
- No clear separation of concerns

With organisation (recommended):

```
src/
  components/
    Button.jsx
    Card.jsx
    Navbar.jsx
  pages/
    Home.jsx
    About.jsx
  utils/
    helpers.js
    constants.js
  App.jsx
  main.jsx
```

Benefits

Easy to find, reuse, and test each component. Teammates know exactly where things live.

Creating a Component in Its Own File

src/components/Car.jsx:

```jsx
// Car.jsx -- standalone component file
function Car({ brand, color = "blue" }) {
  return (
    <div className="car-card">
      <h2>My {color} {brand}</h2>
    </div>
  );
}

// Must export to use in other files
export default Car;
```

src/App.jsx:

```jsx
// Import using relative path
import Car from './components/Car';

function App() {
  return (
    <>
      <h1>My Garage</h1>
      <Car brand="Ford" />
      <Car brand="BMW"  color="black" />
      <Car brand="Audi" color="white" />
    </>
  );
}

export default App;
```

Browser Output

My Garage

My blue Ford
My black BMW
My white Audi

W3Schools import pattern:

```jsx
// Named import (if named export)
import { Car } from './components/Car';

// Default import (default export)
import Car from './components/Car';

// Import multiple items
import Car, { CAR_COLORS }
  from './components/Car';
```

Recommended SWE 381 Folder Structure

```
swe381-app/
  node_modules/
  public/
  src/
    components/
      common/
        Button.jsx
        Card.jsx
        Input.jsx
      layout/
        Header.jsx
        Footer.jsx
        Sidebar.jsx
    pages/
      Home.jsx
      Courses.jsx
      Students.jsx
    utils/
      helpers.js
      constants.js
    App.jsx
    main.jsx
  index.html
```

Folder purposes:

- `components/common/` – small reusable UI pieces (Button, Card, Input)
- `components/layout/` – page structure (Header, Footer, Sidebar)
- `pages/` – full pages composed from components
- `utils/` – helper functions, constants, no JSX
- `App.jsx` – top-level component with routing
- `main.jsx` – entry point, mounts App into DOM

One component per file

Each `.jsx` file should contain **one default-exported component**. Keep related helpers as named exports in the same file.

Outline

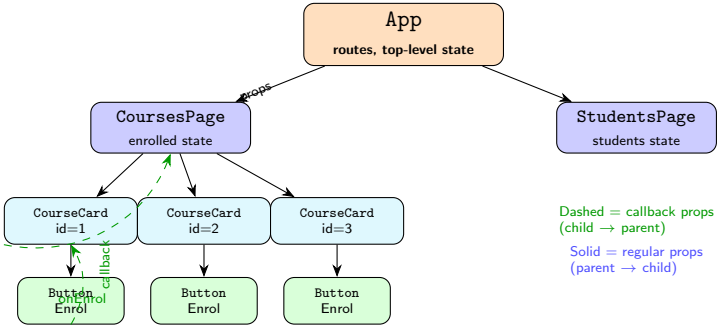
Putting It All Together (1/2)

```jsx
// src/components/common/Button.jsx
function Button({ label, onClick, variant = "primary" }) {
  return (
    <button
      className={`btn btn-${variant}`}
      onClick={onClick}>
      {label}
    </button>
  );
}
export default Button;

// src/components/CourseCard.jsx
import Button from './common/Button';

function CourseCard({ course, onEnrol }) {
  return (
    <div className="card">
      <h3>{course.title}</h3>
      <p>Credits: {course.credits}</p>
      <Button
        label="Enrol"
        onClick={() => onEnrol(course.id)}
      />
    </div>
  );
}
export default CourseCard;
```

Putting It All Together (2/2)

```jsx
// src/pages/Courses.jsx
import { useState } from 'react';
import CourseCard from '../components/CourseCard';

const courses = [
  { id: 1, title: "Web App Dev",   credits: 3 },
  { id: 2, title: "SW Design",     credits: 3 },
  { id: 3, title: "AI Basics",     credits: 2 },
];

function CoursesPage() {
  const [enrolled, setEnrolled] = useState([]);
  const handleEnrol = (id) => {
    if (!enrolled.includes(id)) setEnrolled([...enrolled, id]);
  };
  return (
    <div>
      <h1>Courses</h1>
      {courses.map(c => (
        <CourseCard key={c.id} course={c} onEnrol={handleEnrol} />
      ))}
      <p>Enrolled in {enrolled.length} course(s).</p>
    </div>
  );
}
export default CoursesPage;
```

Component Architecture Diagram



Outline

Lecture 3 – Key Takeaways

1. Functional components are **JavaScript functions** that return JSX – names must start with a capital letter
2. **Props** pass data from parent to child like function arguments; received as an object
3. **Destructuring** removes repetitive `props.x` syntax; set **defaults** with { `prop = value` }
4. **Props are read-only** – children must never modify incoming props; use state instead
5. The `children` **prop** receives JSX placed between a component's opening and closing tags
6. React events use **camelCase**: `onClick`, `onChange`, `onSubmit`
7. Pass function references (not calls): `onClick={fn}` not `onClick={fn()}`
8. The **callback props pattern** lets children communicate back to parents
9. Organise files: one component per `.jsx` file in a `src/components/` folder

Next Lecture: React State – `useState`, lifting state, and controlled components

Practice Exercises

1. Create a `Greeting` component that accepts `name` and `role` props and renders "Hello, Ahmed! You are a Student.". Use **destructuring** in the function signature.
2. Add a **default value** for the `role` prop so it shows "Guest" when not provided. Test with and without the prop.
3. Build a `Card` component that uses the `children` **prop** as a generic container. Wrap a heading and a paragraph inside it in `App.jsx`.
4. Create a `LikeButton` component with an `onClick` prop. In the parent, track how many times it has been clicked using `useState` and display the count.
5. Build a `SearchBar` component with a text input. Use `onChange` to lift the search value to the parent, which displays it in real time below the input.
6. Create a `StudentItem` component that shows a name and a "Remove" button. Use the **callback props pattern**: the parent holds the student list; clicking "Remove" calls the parent's delete handler.
7. **Challenge:** Reorganise your components into a proper `src/components/` folder structure. Split `Button`, `Card`, and `StudentItem` into separate `.jsx` files with proper `import`/`export`.